UJIAN TENGAH SEMESTER (UTS)
PERAMALAN INDEKS PM10 HARIAN DI STASIUN DKI 1 BUNDARAN HI JAKARTA MENGGUNAKAN RECURRENT NEURAL NETWORK (RNN)

Dosen Pengampu : Dr. Dra. Dwina Kuswardani, M.Kom.

Disusun Oleh:
ANGELICA JECKLYNE BEAUTY ANSANAY
202432104
KELAS C

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS TEKNOLOGI DAN BISNIS ENERGI
INSTITUT TEKNOLOGI PLN
2026

DAFTAR ISI

BAB 1 2
1.1 Latar Belakang Masalah 2
1.2 Tujuan dan Alasan Pemilihan Algoritma 2
1.3 Dataset Kaggle 3
1.3.1 Atribut dataset 4
1.3.2 Statistik deskriptif dan kualitas data 6
BAB 2 8
2.1 Konsep Dasar Recurrent Neural Network 8
2.2 Alur Arsitektur 8
2.3 Rumus Standar RNN 9
2.4 Perhitungan Manual (1 Input dan 1 Output) 10
2.5 Mekanisme Pelatihan dan Evaluasi 12
BAB 3 14
3.1 Gambaran Umum Implementasi 14
3.2 Cell 1: Import Library 14
3.3 Cell 2: Load dan Pra-pemrosesan Dataset 16
3.4 Cell 3: Perancangan Arsitektur Model 21
3.5 Cell 4: Hyperparameter dan Proses Pelatihan 23
3.6 Cell 5: Visualisasi Performa Model dan Evaluasi 27
3.6.1 Grafik riwayat training (loss terhadap epoch) 31
3.6.2 Grafik data aktual vs prediksi 32
BAB 4 34
4.1 Keterkaitan Teori dan Implementasi 34
4.2 Kesimpulan 34
4.3 Saran 35

BAB 1
LATAR BELAKANG MASALAH DAN EKSPLORASI DATASET

1.1 Latar Belakang Masalah

Sebagai kota besar dengan aktivitas transportasi dan industri yang padat, Jakarta menghadapi masalah kualitas udara yang berulang. Salah satu polutan yang dipantau setiap hari adalah PM10, yaitu partikel udara berdiameter 10 mikrometer atau lebih kecil yang dapat terhirup hingga ke saluran pernapasan. Di Indonesia, kualitas udara dilaporkan dalam bentuk Indeks Standar Pencemar Udara (ISPU). Setiap polutan dikonversi menjadi sebuah sub-indeks, dan nilai ISPU harian ditentukan oleh sub-indeks yang tertinggi. Polutan penentu nilai tertinggi itu disebut parameter kritis.

Pada dataset yang digunakan, PM10 menjadi parameter kritis pada 2.002 dari 5.173 hari pengamatan (38,7%), lebih sering dibandingkan polutan lain seperti O_3 (1.703 hari), PM2,5 (1.164 hari), SO_2 (123 hari), dan CO (67 hari). Dengan demikian, PM10 adalah polutan yang paling sering menentukan status kualitas udara harian di stasiun ini.

Nilai PM10 dari hari ke hari merupakan deret waktu: nilai hari ini berkaitan dengan nilai hari-hari sebelumnya karena kondisi atmosfer dan sumber emisi berubah secara bertahap. Model yang memperlakukan setiap hari sebagai sampel yang berdiri sendiri akan kehilangan informasi urutan ini. Deep Learning berbasis sekuens seperti RNN dirancang untuk memanfaatkan urutan tersebut. Hasil peramalan dapat menjadi dasar peringatan dini dan perencanaan, misalnya imbauan untuk mengurangi aktivitas luar ruang pada hari yang diperkirakan memburuk. Sesuai ketentuan ujian, NIM 202432104 berakhiran angka genap (4), sehingga topik yang dikerjakan adalah Forecasting Berbasis Time-Series.

1.2 Tujuan dan Alasan Pemilihan Algoritma

Tujuan yang ingin dicapai dalam proyek ini adalah:

1. memprediksi indeks PM10 harian pada hari berikutnya berdasarkan data 30 hari sebelumnya di Stasiun DKI 1 Bundaran HI;
2. memahami dan membuktikan alur komputasi satu langkah waktu RNN melalui perhitungan manual dengan 1 input dan 1 output;
3. mengimplementasikan RNN menggunakan PyTorch, lalu mengevaluasinya pada data uji dengan MAE, RMSE, dan R^2, serta membandingkannya dengan baseline sederhana.

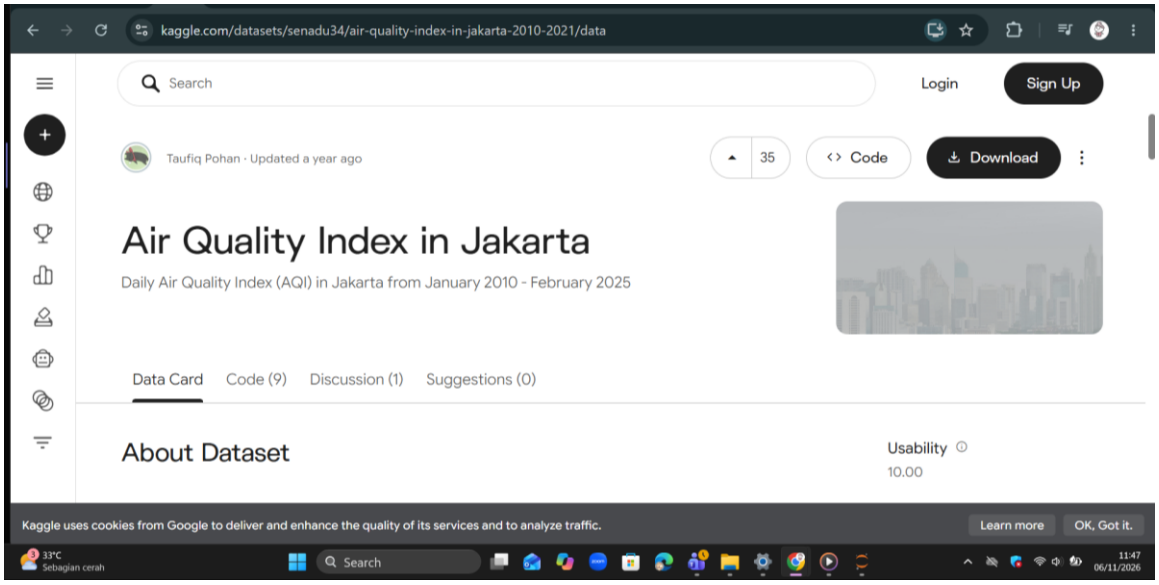
Algoritma yang dipilih adalah **RNN**. Alasannya: (1) RNN memang dirancang untuk data sekuensial karena *hidden state* membawa informasi dari langkah waktu sebelumnya ke langkah berikutnya; (2) arsitekturnya sederhana, hanya satu persamaan pembaruan dan 4.353 parameter pada model ini, sehingga alur komputasinya mudah dibuktikan lewat perhitungan manual; dan (3) data berupa deret harian univariat dengan jendela waktu yang pendek (30 hari), sehingga masalah ketergantungan jangka panjang yang menjadi kelemahan RNN tidak terlalu dominan.

1.3 Dataset Kaggle

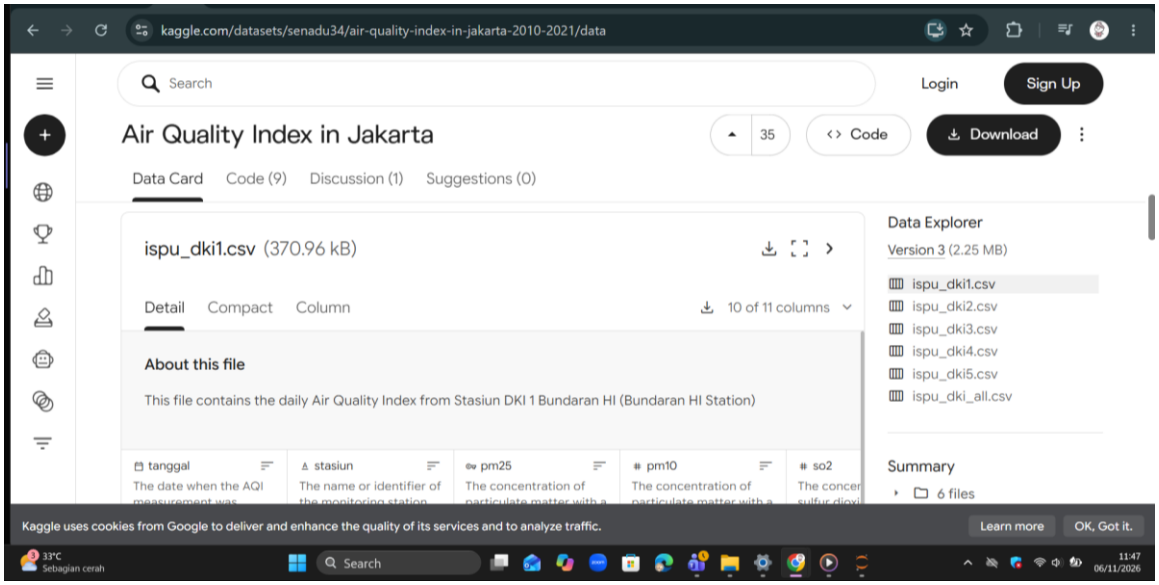
Dataset diambil dari Kaggle dan dipilih secara mandiri. Informasi dataset dirangkum pada Tabel 1.1, sedangkan bukti validasi di situs Kaggle ditampilkan pada Gambar 1.1 dan Gambar 1.2.

Tabel 1.1 Informasi dataset

Nama dataset	Air Quality Index in Jakarta
Pemilik	Taufiq Pohan (akun Kaggle senadu34)
Link Kaggle	https://www.kaggle.com/datasets/senadu34/air-quality-index-in-jakarta-2010-2021
Deskripsi di Kaggle	Daily Air Quality Index (AQI) in Jakarta from January 2010 - February 2025
Isi dataset	6 berkas CSV: ispu_dki1.csv sampai ispu_dki5.csv (satu berkas per stasiun) dan ispu_dki_all.csv (gabungan)
Berkas yang digunakan	ispu_dki1.csv (Stasiun DKI 1 Bundaran HI), 370,96 kB
Jumlah baris dan fitur	5.174 baris × 11 kolom
Rentang waktu	1 Januari 2010 s.d. 28 Februari 2025 (data harian)
Skor usability Kaggle	10,00
Tanggal akses	6 November 2026



Gambar 1.1 Halaman dataset *Air Quality Index in Jakarta* di Kaggle (bukti validasi dan klaim dataset)



Gambar 1.2 Tab *Data* pada dataset yang sama, menampilkan 6 berkas dan kolom pada ispu_dki1.csv

1.3.1 Atribut dataset

Berkas ispu_dki1.csv memiliki 11 atribut yang dijelaskan pada Tabel 1.2. Kolom polutan (pm25, pm10, so2, co, o3, no2) berisi **sub-indeks ISPU**, bukan konsentrasi dalam µg/m^3. Hal ini terlihat dari nilai kolom polutan yang sama persis dengan kolom max pada 4.957

dari 4.961 baris yang datanya lengkap. Karena itu, sumbu grafik pada laporan ini diberi label "Indeks PM10 (ISPU)".

Tabel 1.2 Atribut pada ispu_dki1.csv

No	Atribut	Tipe	Keterangan	Kosong / Rentang
1	tanggal	Tanggal	Tanggal pengukuran harian	0 kosong
2	stasiun	Teks	Nama stasiun pemantau; seluruh baris bernilai "DKI1 (Bunderan HI)"	0 kosong
3	pm25	Numerik	Sub-indeks partikel PM2,5	4.027 kosong; 10–122
4	**pm10**	Numerik	**Sub-indeks partikel PM10 (target peramalan)**	216 kosong; 4–104
5	so2	Numerik	Sub-indeks sulfur dioksida	188 kosong; 1–112
6	co	Numerik	Sub-indeks karbon monoksida	168 kosong; 3–95
7	o3	Numerik	Sub-indeks ozon	209 kosong; 3–198
8	no2	Numerik	Sub-indeks nitrogen dioksida	199 kosong; 1–98
9	max	Numerik	Nilai ISPU tertinggi pada hari tersebut	13 kosong; 0–198
10	critical	Teks	Parameter kritis, yaitu polutan penentu nilai max (PM10, O3, PM25, SO2, CO)	114 kosong
11	categori	Teks (label)	Kategori kualitas udara: BAIK, SEDANG, TIDAK SEHAT, TIDAK ADA DATA	0 kosong

Sebaran label pada kolom categori adalah SEDANG 3.684 baris, BAIK 1.189 baris, TIDAK SEHAT 186 baris, dan TIDAK ADA DATA 114 baris. Label ini tidak dipakai sebagai target karena proyek ini adalah peramalan nilai numerik PM10, bukan klasifikasi.

1.3.2 Statistik deskriptif dan kualitas data

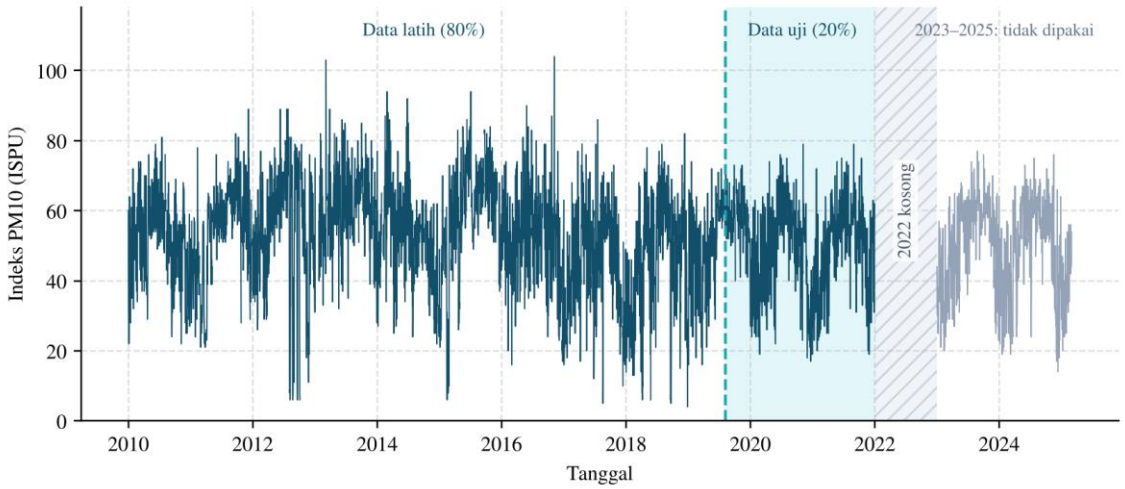
Tabel 1.3 Statistik deskriptif kolom pm10 (seluruh berkas, 4.957 nilai terisi)

Jumlah	Rata-rata	Std. deviasi	Min	Q1	Median	Q3	Maks
4.957	51,85	14,40	4	43	54	62	104

Ada tiga catatan kualitas data yang memengaruhi pemodelan:

- **Kolom pm25 hampir kosong** (4.027 dari 5.173 baris, atau 77,8%), sehingga tidak dipakai. Target dipilih pm10 yang jauh lebih lengkap.
- **Seluruh tahun 2022 tidak memiliki data.** Dari 5.538 hari kalender pada rentang dataset, hanya 5.173 hari yang memiliki baris, dan 365 hari yang hilang adalah tepat seluruh tahun 2022. Mengisi satu tahun penuh dengan interpolasi akan menciptakan data buatan, sehingga pemodelan memakai segmen kontinu **1 Januari 2010 s.d. 31 Desember 2021 (4.383 hari)**.
- **Celah pendek diisi dengan interpolasi linear.** Pada segmen 2010–2021 terdapat 212 hari tanpa nilai PM10 yang tersebar pada 88 celah, dengan celah terpanjang 20 hari.

Data dibagi secara kronologis: 80% pertama (3.506 hari, 1 Januari 2010 s.d. 7 Agustus 2019) sebagai data latih dan 20% terakhir (877 hari, 8 Agustus 2019 s.d. 31 Desember 2021) sebagai data uji. Pada data yang digunakan, data latih memiliki rata-rata 52,7 dan simpangan baku 15,0, sedangkan data uji memiliki rata-rata 50,5 dan simpangan baku 12,4. Gambar 1.3 menunjukkan seluruh deret PM10 beserta pembagian tersebut. Nilai PM10 berfluktuasi naik-turun di sekitar 40–70 sepanjang periode tanpa tren jangka panjang yang tegas, dengan beberapa lonjakan hingga 104 dan penurunan tajam ke kisaran 4–6.



Gambar 1.3 Deret waktu PM10 harian Stasiun DKI 1 Bundaran HI beserta pembagian data latih dan data uji; tahun 2022 tidak memiliki data

BAB 2
PEMAHAMAN ALGORITMA DAN PERHITUNGAN MANUAL

2.1 Konsep Dasar Recurrent Neural Network

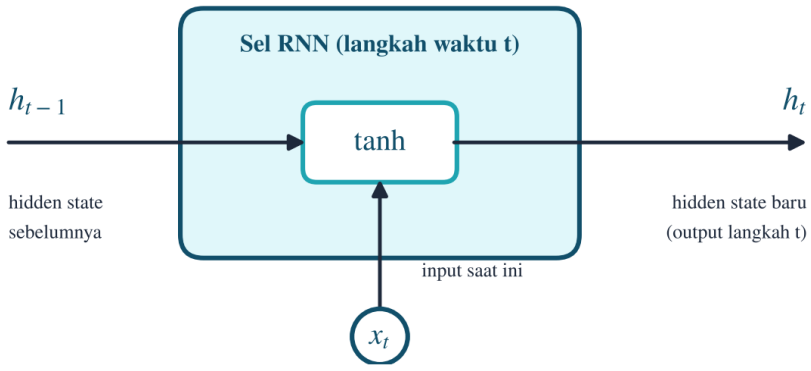
Jaringan saraf *feed-forward* biasa memproses setiap input secara terpisah, sehingga tidak memiliki ingatan tentang input sebelumnya. *Recurrent Neural Network* (RNN) menambahkan koneksi berulang: keluaran sel pada langkah waktu t−1, yang disebut *hidden state* h_{t-1}, dimasukkan kembali sebagai input pada langkah waktu t (Elman, 1990). Dengan cara ini, h_t berfungsi sebagai memori ringkas dari seluruh urutan yang sudah dibaca.

Satu set bobot yang sama (W_x, W_h, dan b) dipakai berulang pada setiap langkah waktu, sehingga jumlah parameter tidak bergantung pada panjang urutan. Pelatihan dilakukan dengan *Backpropagation Through Time* (BPTT), yaitu jaringan dibentangkan (*unrolled*) sepanjang langkah waktu lalu gradien dihitung mundur (Rumelhart dkk., 1986). Pada urutan yang panjang, gradien dapat mengecil (*vanishing*) atau membesar (*exploding*) sehingga RNN sulit mengingat informasi yang jauh (Pascanu dkk., 2013). Pada proyek ini hal tersebut ditangani dengan jendela waktu yang pendek (30 hari) dan pemotongan gradien (*gradient clipping*).

2.2 Alur Arsitektur

Gambar 2.1 menunjukkan satu sel RNN. Sel menerima dua masukan, yaitu input saat ini x_t dan hidden state sebelumnya h_{t-1}. Keduanya dikalikan bobot masing-masing, dijumlahkan dengan bias, lalu dilewatkan fungsi aktivasi tanh sehingga menghasilkan hidden state baru h_t.

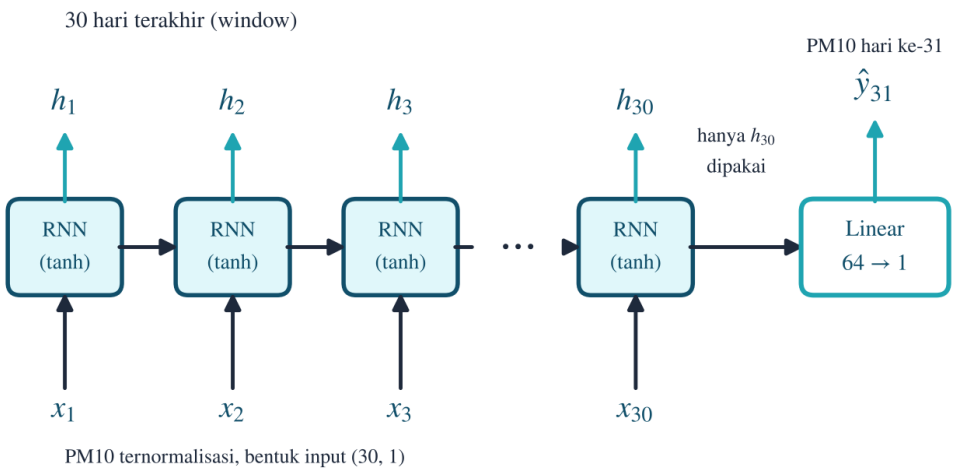
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$



Gambar 2.1 Satu sel RNN pada langkah waktu t

Pada implementasi, sel yang sama dijalankan berulang selama 30 langkah (Gambar 2.2). Jenis arsitekturnya adalah *many-to-one*: seluruh urutan dibaca, tetapi hanya hidden state terakhir yang dipakai untuk menghasilkan satu prediksi. Alurnya:

1. Input berupa jendela 30 hari PM10 ternormalisasi (x_1, …, x_{30}) dengan bentuk (30, 1).
2. Hidden state awal h_0 bernilai nol (nilai bawaan nn.RNN).
3. Pada setiap langkah t, sel menggabungkan x_t dan h_{t-1} menjadi h_t berukuran 64 nilai (64 neuron tersembunyi).
4. Setelah langkah ke-30, h_{30} menjadi ringkasan seluruh jendela.
5. Lapisan linear (64 → 1) memetakan h_{30} menjadi satu nilai prediksi, yaitu PM10 hari ke-31 yang masih ternormalisasi.



Gambar 2.2 RNN yang dibentangkan sepanjang 30 langkah waktu, dengan lapisan linear pada keluaran akhir

2.3 Rumus Standar RNN

Pembaruan *hidden state* pada RNN mengikuti rumus berikut:

$$h_t = \tanh\left(W_x \cdot x_t + W_h \cdot h_{t-1} + b\right)$$

Tabel 2.1 Notasi pada rumus RNN

Simbol	Arti	Dimensi pada model
x_t	Input pada langkah waktu t (PM10 ternormalisasi)	1 × 1
h_{t-1}	Hidden state langkah sebelumnya	64 × 1
h_t	Hidden state baru (output sel)	64 × 1

Simbol	Arti	Dimensi pada model
W_x	Bobot input	64 × 1
W_h	Bobot hidden state (koneksi berulang)	64 × 64
b	Bias (di PyTorch merupakan jumlah dua bias, b_ih + b_hh)	64 × 1
tanh	Fungsi aktivasi; membatasi nilai pada rentang (−1, 1)	-

Setelah langkah terakhir, lapisan linear menghasilkan prediksi $\hat{y} = W_y \cdot h_T + b_y$, dengan W_y berukuran 1 × 64. Jumlah parameter model: RNN memiliki 64 + 4.096 + 64 + 64 = 4.288 parameter dan lapisan linear 65 parameter, sehingga total **4.353 parameter**.

2.4 Perhitungan Manual (1 Input dan 1 Output)

Perhitungan manual berikut membuktikan alur komputasi internal RNN pada **satu langkah waktu** dengan **satu nilai input (x_t)** dan **satu nilai output (h_t)**. Agar mudah dihitung dengan tangan, digunakan satu neuron tersembunyi dan bobot contoh yang sederhana. Bobot ini bukan hasil pelatihan, karena model sebenarnya memiliki 64 neuron dan bobot hasil pelatihan.

Tabel 2.2 Nilai yang digunakan pada perhitungan manual

Besaran	Nilai	Keterangan
PM10 hari ini	60	Contoh input pada skala ISPU
x_{min}, x_{max}	4 dan 104	Minimum dan maksimum data latih (dipakai pada normalisasi di kode)
W_x	0,8	Bobot input (contoh)
W_h	0,5	Bobot hidden state (contoh)
b	0,1	Bias (contoh)
h_{t-1}	0,2	Hidden state sebelumnya (contoh)
W_y, b_y	0,9 dan 0,02	Bobot dan bias lapisan keluaran (contoh)

Langkah 1: normalisasi input. Input dinormalisasi dengan Min-Max agar berada pada rentang 0–1, sama seperti yang dilakukan pada kode.

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

$$x_t = \frac{60 - 4}{104 - 4} = 0{,}56$$

Langkah 2: hitung suku input dan suku memori.

- Suku input: $W_x \cdot x_t = 0{,}8 \times 0{,}56 =$ **0,448**
- Suku memori: $W_h \cdot h_{t-1} = 0{,}5 \times 0{,}20 =$ **0,100**

Langkah 3: jumlahkan dengan bias (nilai sebelum aktivasi, z).

$$z = (0{,}8)(0{,}56) + (0{,}5)(0{,}20) + 0{,}1 = 0{,}448 + 0{,}100 + 0{,}100 = 0{,}648$$

Langkah 4: terapkan fungsi aktivasi tanh untuk memperoleh hidden state baru h_t.

$$h_t = \tanh(0{,}648) = \frac{e^{2(0{,}648)} - 1}{e^{2(0{,}648)} + 1} = \frac{3{,}6546 - 1}{3{,}6546 + 1} = \frac{2{,}6546}{4{,}6546} = 0{,}5703$$

Jadi, $\mathbf{h_t = 0{,}5703}$. Hasil ini diverifikasi dengan modul nn.RNN milik PyTorch menggunakan bobot dan input yang sama, dan keluarannya 0,57032 sehingga sama dengan hasil hitung manual.

Langkah 5: keluaran lapisan linear dan pengembalian ke skala asli.

$$\hat{y}_{norm} = W_y \cdot h_t + b_y = (0{,}9)(0{,}5703) + 0{,}02 = 0{,}5333$$

$$\hat{y} = 0{,}5333 \times (104 - 4) + 4 = 57{,}33$$

Tabel 2.3 Ringkasan hasil perhitungan manual

Tahap	Hasil	Keterangan
x_t (setelah normalisasi)	0,56	PM10 = 60
$W_x \cdot x_t$	0,448	Kontribusi input
$W_h \cdot h_{t-1}$	0,100	Kontribusi memori
$z = W_x x_t + W_h h_{t-1} + b$	0,648	Sebelum aktivasi
$h_t = \tanh(z)$	**0,5703**	**Output sel RNN**
Prediksi ternormalisasi	0,5333	Setelah lapisan linear
Prediksi skala ISPU	57,33	Setelah denormalisasi

Interpretasi. Input yang cukup tinggi (0,56) menggeser hidden state naik dari 0,20 menjadi 0,5703, sedangkan tanh menjaga nilainya tetap berada di antara −1 dan 1. Suku memori (0,100) menunjukkan bahwa informasi hari sebelumnya ikut menentukan hasil, dan inilah yang membedakan RNN dari jaringan biasa. Prediksi 57,33 dekat dengan input 60 karena bobot contoh dipilih untuk ilustrasi. Pada model sebenarnya, perhitungan yang sama dilakukan oleh 64 neuron selama 30 langkah waktu dengan bobot hasil pelatihan.

2.5 Mekanisme Pelatihan dan Evaluasi

Pelatihan model terdiri dari beberapa tahap yang diulang pada setiap *mini-batch*:

1. **Forward propagation:** jendela 30 hari dilewatkan melalui RNN dan lapisan linear untuk menghasilkan prediksi.
2. **Fungsi loss:** selisih antara prediksi dan nilai aktual diukur dengan *Mean Squared Error* (MSE), yaitu rata-rata kuadrat selisih. Kuadrat membuat kesalahan besar dihukum lebih berat.
3. **Backpropagation Through Time:** gradien MSE terhadap seluruh bobot dihitung mundur sepanjang 30 langkah waktu. Gradien dipotong agar normanya tidak lebih dari 1 untuk mencegah *exploding gradient*.
4. **Optimizer:** algoritma Adam (Kingma & Ba, 2015) memperbarui bobot menggunakan gradien dengan laju pembelajaran yang disesuaikan untuk tiap parameter.

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Performa pada data uji diukur dengan tiga metrik pada skala asli (indeks ISPU):

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

- **MAE** adalah rata-rata besar kesalahan absolut, mudah ditafsirkan karena satuannya sama dengan indeks PM10.
- **RMSE** juga bersatuan indeks PM10 tetapi lebih peka terhadap kesalahan besar.

- **R^2** menyatakan proporsi variasi data aktual yang dapat dijelaskan model; nilai 1 berarti sempurna, sedangkan 0 sama dengan menebak rata-rata.

Sebagai pembanding digunakan *baseline* sederhana, yaitu memprediksi nilai hari ini sama dengan nilai kemarin. Model yang belajar dengan benar seharusnya mengalahkan baseline ini.

BAB 3
IMPLEMENTASI PRAKTIKUM DAN PENJELASAN BARIS KODE

3.1 Gambaran Umum Implementasi

Implementasi ditulis dalam satu notebook Jupyter (.ipynb) menggunakan Python dan PyTorch, mengikuti template notebook yang disediakan. Notebook terdiri dari lima cell kode, dan setiap cell diawali komentar watermark #202432104_AngelicaJecklyneBeautyAnsanay sesuai ketentuan. Program dan dataset diunggah ke repositori GitHub publik: [tempel link repositori GitHub di sini].

Tabel 3.1 Tahapan implementasi

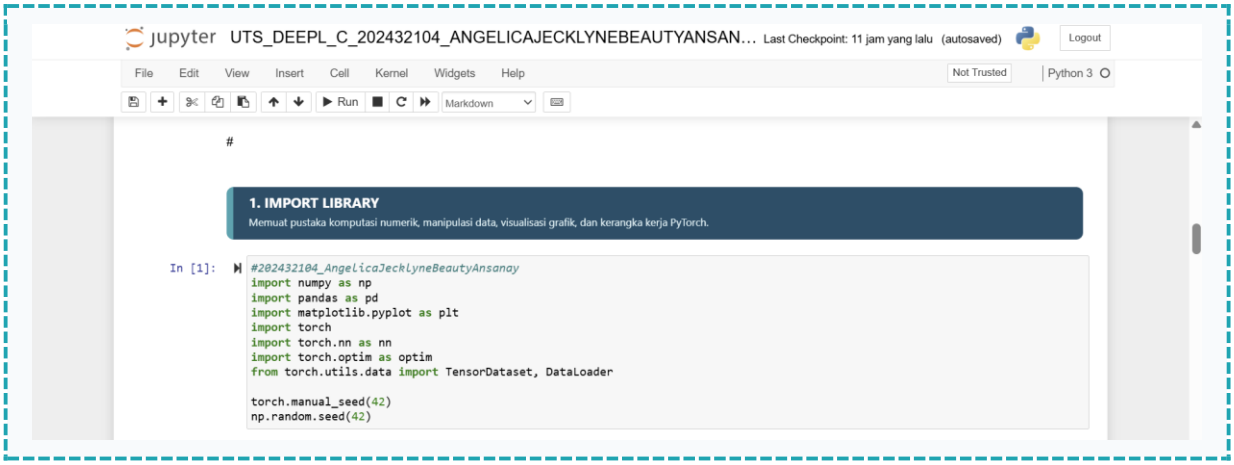
Cell	Tahap	Keterangan
1	Import library	Memuat pustaka numerik, data, grafik, dan PyTorch
2	Load dan preprocessing	Membaca dataset, membersihkan data, normalisasi, sliding window, pembagian data latih dan uji
3	Arsitektur model	Kelas SequenceModel berisi nn.RNN dan nn.Linear
4	Hyperparameter dan training	Konfigurasi pelatihan dan siklus training selama 50 epoch
5	Visualisasi dan evaluasi	Grafik loss, metrik, serta grafik data aktual vs prediksi

3.2 Cell 1: Import Library

Cell ini memuat seluruh pustaka yang dibutuhkan dan mengunci *random seed* agar hasil dapat direproduksi.

Baris	Kode	Penjelasan
1	#202432104_AngelicaJecklyneBeautyAnsanay	Komentar watermark #nim_nama yang diwajibkan dosen. Baris komentar diabaikan saat program dijalankan.
2	import numpy as np	Mengimpor NumPy dengan alias np untuk operasi array dan perhitungan numerik (membentuk jendela data dan menghitung metrik).

Baris	Kode	Penjelasan
3	import pandas as pd	Mengimpor pandas dengan alias pd untuk membaca berkas CSV dan mengolah data deret waktu berindeks tanggal.
4	import matplotlib.pyplot as plt	Mengimpor modul pyplot dari matplotlib dengan alias plt untuk membuat grafik.
5	import torch	Mengimpor PyTorch, yaitu pustaka inti yang menyediakan tensor dan perhitungan gradien otomatis.
6	import torch.nn as nn	Mengimpor torch.nn dengan alias nn, yang berisi layer jaringan (nn.RNN, nn.Linear) dan fungsi loss (nn.MSELoss).
7	import torch.optim as optim	Mengimpor torch.optim dengan alias optim, yang berisi algoritma optimasi seperti Adam.
8	from torch.utils.data import TensorDataset, DataLoader	Mengimpor TensorDataset (membungkus pasangan input dan target) dan DataLoader (membagi data menjadi *mini-batch* dan mengacaknya).
10	torch.manual_seed(42)	Mengunci *seed* generator acak PyTorch pada 42 sehingga inisialisasi bobot dan pengacakan batch sama pada setiap eksekusi.
11	np.random.seed(42)	Mengunci *seed* generator acak NumPy pada 42 dengan tujuan yang sama.



Gambar 3.1 Screenshot Jupyter: Cell 1 (import library)

3.3 Cell 2: Load dan Pra-pemrosesan Dataset

Cell ini membaca dataset Kaggle, membersihkannya, menormalisasi nilai PM10, membentuk jendela geser (*sliding window*), lalu membagi data latih dan data uji secara kronologis.

Baris	Kode	Penjelasan
1	#202432104_AngelicaJecklyneBeautyAnsanay	Komentar watermark #nim_nama.
2	# 1. Muat dataset (ubah ke URL raw GitHub jika dijalankan dari repositori)	Komentar tahap 1: memuat dataset. Jika notebook dijalankan dari repositori, path dapat diganti dengan URL *raw* GitHub.
3	DATA_PATH = 'ispu_dki1.csv'	Menyimpan nama berkas CSV pada variabel DATA_PATH agar mudah diganti.
4	df = pd.read_csv(DATA_PATH, parse_dates=['tanggal'])	Membaca CSV menjadi DataFrame df. Argumen parse_dates=['tanggal'] mengubah kolom tanggal menjadi tipe tanggal sehingga dapat diurutkan dan dijadikan indeks waktu.
5	print('Ukuran dataset :', df.shape)	Mencetak ukuran dataset dengan df.shape, yaitu (5173, 11): 5.173 baris dan 11 kolom.

Baris	Kode	Penjelasan
6	print('Rentang tanggal :', df['tanggal'].min().date(), 's.d.', df['tanggal'].max().date())	Mencetak tanggal paling awal dan paling akhir. .min() dan .max() mengambil nilai terkecil dan terbesar, .date() membuang bagian jam.
7	print('PM10 kosong :', df['pm10'].isna().sum())	Menghitung nilai kosong pada kolom pm10 dengan isna().sum(). Hasilnya 216.
9	# 2. Pembersihan & pra-pemrosesan (target: PM10 harian)	Komentar tahap 2: pembersihan dan pra-pemrosesan dengan target PM10 harian.
10	series = df.sort_values('tanggal').set_index('tanggal')['pm10']	Mengurutkan data berdasarkan tanggal (sort_values), menjadikan tanggal sebagai indeks (set_index), lalu mengambil kolom pm10 sehingga terbentuk deret waktu series.
11	series = series.reindex(pd.date_range(series.index.min(), series.index.max(), freq='D'))	pd.date_range(..., freq='D') membuat kalender harian lengkap dari tanggal awal sampai akhir. reindex menyelaraskan deret ke kalender itu sehingga hari yang tidak punya baris (seluruh tahun 2022) muncul sebagai nilai kosong. Interval waktu yang konsisten diperlukan oleh jendela geser.
12	series = series.loc[:'2021-12-31']	Memotong deret sampai 31 Desember 2021. Tahun 2022 kosong penuh (365 hari), sehingga tidak diisi dengan data buatan.
13	print('Hari tanpa data 2010-2021:', series.isna().sum())	Mencetak sisa hari tanpa data pada 2010–2021, yaitu 212 hari.

Baris	Kode	Penjelasan
14	series = series.interpolate(method='linear', limit_direction='both')	Mengisi nilai kosong dengan interpolasi linear, yaitu garis lurus antara nilai sebelum dan sesudah celah. limit_direction='both' memastikan celah di ujung deret juga terisi. Celah terpanjang hanya 20 hari.
16	window = 30	Menetapkan panjang jendela window = 30: model melihat 30 hari terakhir untuk memprediksi hari ke-31.
17	train_size = int(len(series) * 0.8)	Menghitung jumlah data latih sebesar 80% dari panjang deret: 4.383 × 0,8 = 3.506,4, dibulatkan ke bawah menjadi 3.506 hari.
18–19	data_min = series.iloc[:train_size].min() data_max = series.iloc[:train_size].max()	Mencari nilai minimum (4,0) dan maksimum (104,0) **hanya dari data latih**. Dengan begitu tidak ada informasi dari data uji yang bocor ke proses normalisasi.
20	scaled = ((series - data_min) / (data_max - data_min)).values.astype('float32')	Normalisasi Min-Max: (series - data_min) / (data_max - data_min) mengubah nilai ke rentang 0–1. .values mengambil array NumPy dan .astype('float32') menyamakan tipe data dengan bawaan PyTorch.
22	def create_sequences(data, window):	Mendefinisikan fungsi create_sequences untuk membentuk jendela geser dari deret data dengan panjang window.
23	X, y = [], []	Menyiapkan dua daftar kosong: X untuk input dan y untuk target.

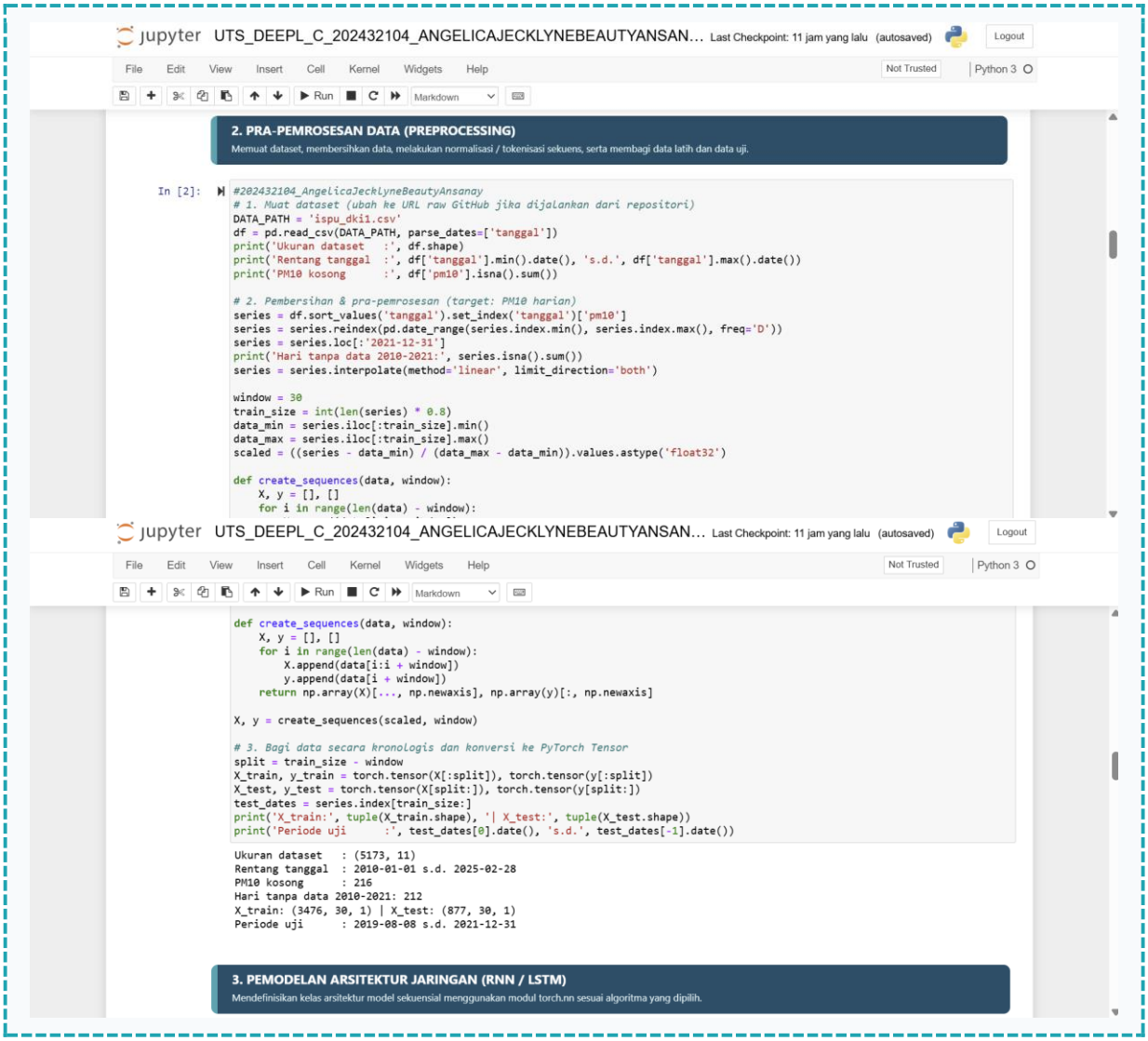
Baris	Kode	Penjelasan
24	for i in range(len(data) - window):	Perulangan sepanjang len(data) - window posisi awal, agar jendela beserta targetnya tidak melewati ujung data.
25	X.append(data[i:i + window])	Menambahkan 30 nilai berurutan data[i : i + window] sebagai satu sampel input.
26	y.append(data[i + window])	Menambahkan nilai tepat setelah jendela, data[i + window], sebagai target (PM10 hari ke-31).
27	return np.array(X)[..., np.newaxis], np.array(y)[:, np.newaxis]	Mengubah daftar menjadi array NumPy. [..., np.newaxis] menambah dimensi fitur sehingga X berbentuk (sampel, 30, 1), sesuai input nn.RNN dengan batch_first=True. y diubah menjadi bentuk (sampel, 1).
29	X, y = create_sequences(scaled, window)	Menjalankan fungsi pada data ternormalisasi. Hasilnya 4.353 sampel (4.383 − 30).
31	# 3. Bagi data secara kronologis dan konversi ke PyTorch Tensor	Komentar tahap 3: pembagian data secara kronologis dan konversi ke tensor.
32	split = train_size - window	Menghitung titik pemisah sampel: 3.506 − 30 = 3.476. Sampel ke-i memiliki target pada hari ke-(i+30), sehingga 3.476 sampel pertama memiliki target di dalam 3.506 hari data latih.
33–34	X_train, y_train = torch.tensor(X[:split]), torch.tensor(y[:split]) X_test, y_test = torch.tensor(X[split:]), torch.tensor(y[split:])	Memotong X dan y menjadi data latih (3.476 sampel pertama) dan data uji (877 sampel sisanya),

Baris	Kode	Penjelasan
		lalu mengubahnya menjadi tensor PyTorch dengan torch.tensor. Data tidak diacak karena urutan waktu harus dipertahankan. Jendela sampel uji pertama memuat 30 hari terakhir periode latih, tetapi targetnya berada di periode uji.
35	test_dates = series.index[train_size:]	Menyimpan tanggal target data uji (series.index[train_size:]) untuk sumbu-x grafik.
36	print('X_train:', tuple(X_train.shape), '| X_test:', tuple(X_test.shape))	Mencetak bentuk tensor: (3476, 30, 1) untuk data latih dan (877, 30, 1) untuk data uji.
37	print('Periode uji :', test_dates[0].date(), 's.d.', test_dates[-1].date())	Mencetak periode data uji, yaitu 8 Agustus 2019 sampai 31 Desember 2021.

Output yang dihasilkan:

```
Ukuran dataset   : (5173, 11)
Rentang tanggal  : 2010-01-01 s.d. 2025-02-28
PM10 kosong      : 216
Hari tanpa data 2010-2021: 212
X_train: (3476, 30, 1) | X_test: (877, 30, 1)
Periode uji      : 2019-08-08 s.d. 2021-12-31
```

Analisis. Dataset berukuran 5.173 × 11 sesuai dengan yang tertera di Kaggle. Dari 216 nilai PM10 yang kosong, 212 berada pada segmen 2010–2021 dan seluruhnya dapat diisi dengan interpolasi karena celahnya pendek. Data latih berjumlah 3.476 sampel dan data uji 877 sampel (rasio sekitar 80:20). Setiap sampel berbentuk (30, 1), yaitu 30 hari dengan satu fitur, sesuai dengan format input RNN pada Bab 2.



Gambar 3.2 Screenshot Jupyter: Cell 2 (load dan preprocessing) beserta output teksnya

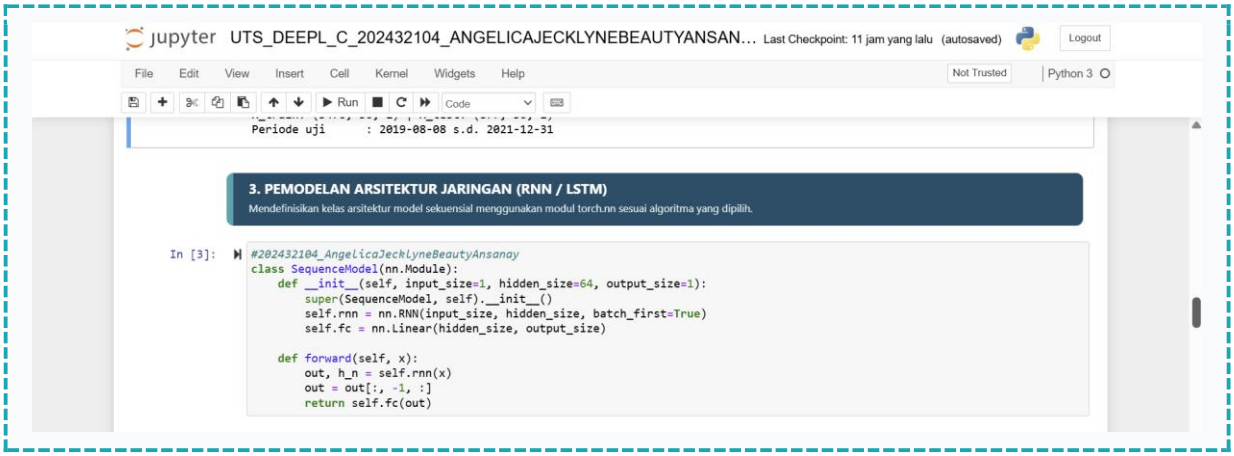
3.4 Cell 3: Perancangan Arsitektur Model

Cell ini mendefinisikan model sebagai kelas PyTorch yang terdiri dari satu lapisan RNN dan satu lapisan linear, sesuai arsitektur pada Gambar 2.2.

Baris	Kode	Penjelasan
1	#202432104_AngelicaJecklyneBeautyAnsanay	Komentar watermark #nim_nama.
2	class SequenceModel(nn.Module):	Mendefinisikan kelas SequenceModel sebagai turunan nn.Module, kelas dasar semua jaringan di PyTorch.
3	def __init__(self, input_size=1, hidden_size=64, output_size=1):	Konstruktor kelas. input_size=1 karena hanya ada satu fitur (PM10), hidden_size=64 adalah

Baris	Kode	Penjelasan
		jumlah neuron tersembunyi, dan output_size=1 karena model memprediksi satu nilai.
4	super(SequenceModel, self).__init__()	Memanggil konstruktor kelas induk agar nn.Module terinisialisasi dan seluruh layer terdaftar sebagai parameter model.
5	self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)	Membuat lapisan RNN (aktivasi tanh bawaan) satu tingkat. batch_first=True membuat bentuk input (batch, waktu, fitur). Parameternya: W_x 64×1, W_h 64×64, serta dua bias 64, total 4.288.
6	self.fc = nn.Linear(hidden_size, output_size)	Membuat lapisan linear 64 → 1 yang memetakan hidden state akhir menjadi satu nilai prediksi (65 parameter).
8	def forward(self, x):	Mendefinisikan forward, yaitu alur maju yang dijalankan setiap kali model(x) dipanggil.
9	out, h_n = self.rnn(x)	Menjalankan RNN pada seluruh 30 langkah. out berisi hidden state setiap langkah dengan bentuk (batch, 30, 64), sedangkan h_n adalah hidden state terakhir (tidak dipakai langsung). Hidden state awal h_0 bernilai nol.
10	out = out[:, -1, :]	Mengambil hidden state langkah terakhir (-1) sehingga bentuknya (batch, 64). Inilah ringkasan seluruh jendela 30 hari pada skema *many-to-one*.
11	return self.fc(out)	Memasukkan hidden state akhir ke lapisan linear dan mengembalikan prediksi berbentuk (batch, 1).

Analisis. Model memiliki total 4.353 parameter (4.288 pada RNN dan 65 pada lapisan linear). Jumlah ini kecil dan sepadan dengan 3.476 sampel latih. Kode pada baris 9 merupakan implementasi langsung rumus $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$ yang dihitung manual pada Bab 2, hanya saja dijalankan oleh 64 neuron selama 30 langkah.



Gambar 3.3 Screenshot Jupyter: Cell 3 (arsitektur model)

3.5 Cell 4: Hyperparameter dan Proses Pelatihan

Cell ini menentukan hyperparameter, membuat model beserta fungsi loss dan optimizer, lalu menjalankan siklus pelatihan selama 50 epoch.

Baris	Kode	Penjelasan
1	#202432104_AngelicaJecklyneBeautyAnsanay	Komentar watermark #nim_nama.
2	# 1. Inisiasi hyperparameter	Komentar tahap 1: inisiasi hyperparameter.
3	epochs = 50	epochs = 50: seluruh data latih dilewatkan ke model sebanyak 50 putaran.
4	learning_rate = 0.01	learning_rate = 0.01: ukuran langkah pembaruan bobot oleh optimizer.
5	batch_size = 64	batch_size = 64: bobot diperbarui setiap 64 sampel.
6	hidden_size = 64	hidden_size = 64: jumlah neuron tersembunyi pada RNN.
8	# 2. Inisiasi model, fungsi loss, optimizer, dan data loader	Komentar tahap 2: inisiasi model, fungsi loss, optimizer, dan data loader.

Baris	Kode	Penjelasan
9	model = SequenceModel(hidden_size=hidden_size)	Membuat objek model dari kelas SequenceModel dengan bobot awal acak (dikunci oleh *seed* 42).
10	criterion = nn.MSELoss()	Memilih MSELoss sebagai fungsi loss, sesuai untuk regresi (peramalan nilai numerik).
11	optimizer = optim.Adam(model.parameters(), lr=learning_rate)	Membuat optimizer Adam yang akan memperbarui seluruh parameter model (model.parameters()) dengan laju pembelajaran 0,01.
12	train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=batch_size, shuffle=True)	Membungkus data latih ke TensorDataset, lalu DataLoader membaginya menjadi *mini-batch* 64 sampel (55 batch per epoch). shuffle=True mengacak urutan sampel setiap epoch; yang diacak hanya urutan jendela, bukan urutan hari di dalam jendela.
14	# 3. Simpan riwayat loss	Komentar tahap 3: menyimpan riwayat loss.
15	train_losses = []	Daftar kosong train_losses untuk menyimpan rata-rata loss latih setiap epoch.
16	test_losses = []	Daftar kosong test_losses untuk loss data uji, dipakai memantau gejala *overfitting*.
18	# 4. Training loop	Komentar tahap 4: *training loop*.
19	for epoch in range(epochs):	Perulangan terluar: satu iterasi sama dengan satu epoch.
20	model.train()	model.train() mengaktifkan mode pelatihan.
21	running_loss = 0.0	Menyiapkan akumulator loss untuk epoch ini.
22	for xb, yb in train_loader:	Perulangan dalam: mengambil satu *mini-batch* input xb dan target yb dari train_loader.

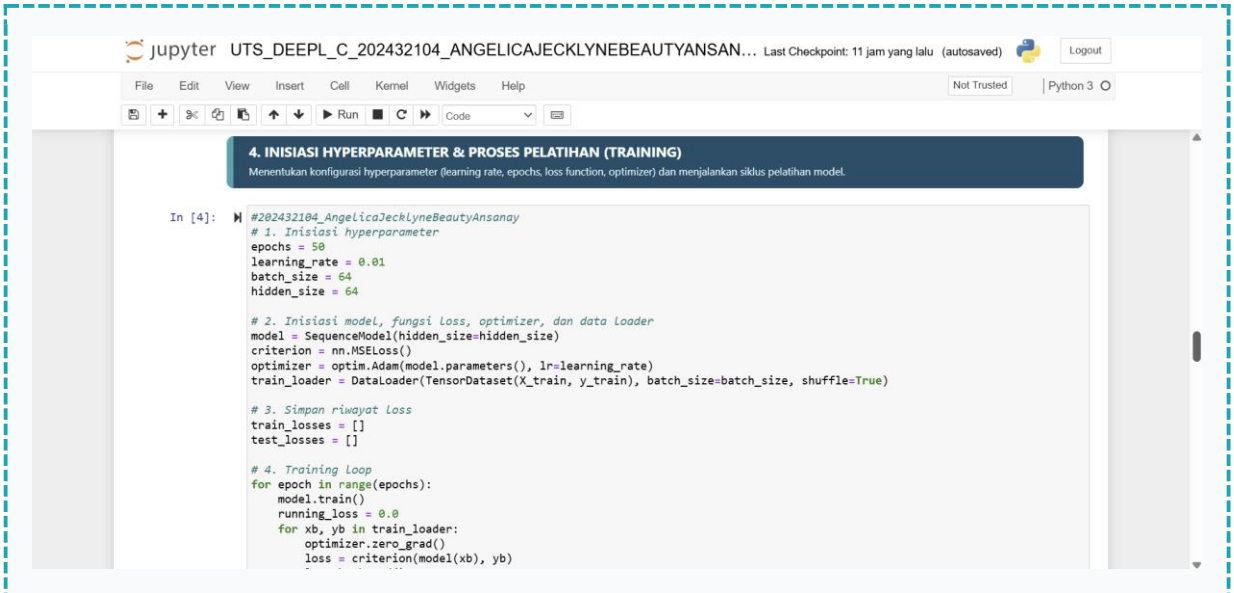
Baris	Kode	Penjelasan
23	optimizer.zero_grad()	optimizer.zero_grad() mengosongkan gradien dari batch sebelumnya, karena PyTorch mengakumulasi gradien secara bawaan.
24	loss = criterion(model(xb), yb)	*Forward propagation*: model(xb) menghasilkan prediksi, lalu criterion menghitung MSE terhadap target yb.
25	loss.backward()	*Backpropagation* (BPTT): loss.backward() menghitung gradien loss terhadap semua bobot sepanjang 30 langkah waktu.
26	nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)	*Gradient clipping*: jika norma gradien lebih dari 1, gradien diskalakan turun. Ini mencegah *exploding gradient* yang umum pada RNN.
27	optimizer.step()	optimizer.step() memperbarui bobot memakai gradien tersebut.
28	running_loss += loss.item() * xb.size(0)	Menambahkan loss batch dikali jumlah sampel batch (xb.size(0)) ke akumulator. Pembobotan ini menjaga rata-rata tetap benar karena batch terakhir lebih kecil. .item() mengubah tensor menjadi angka biasa.
29	train_losses.append(running_loss / len(X_train))	Setelah semua batch selesai, menyimpan rata-rata loss latih epoch ini (akumulator dibagi jumlah sampel latih).
31	model.eval()	model.eval() mengubah model ke mode evaluasi.
32	with torch.no_grad():	torch.no_grad() menonaktifkan perhitungan gradien karena tahap ini hanya mengukur, bukan melatih.

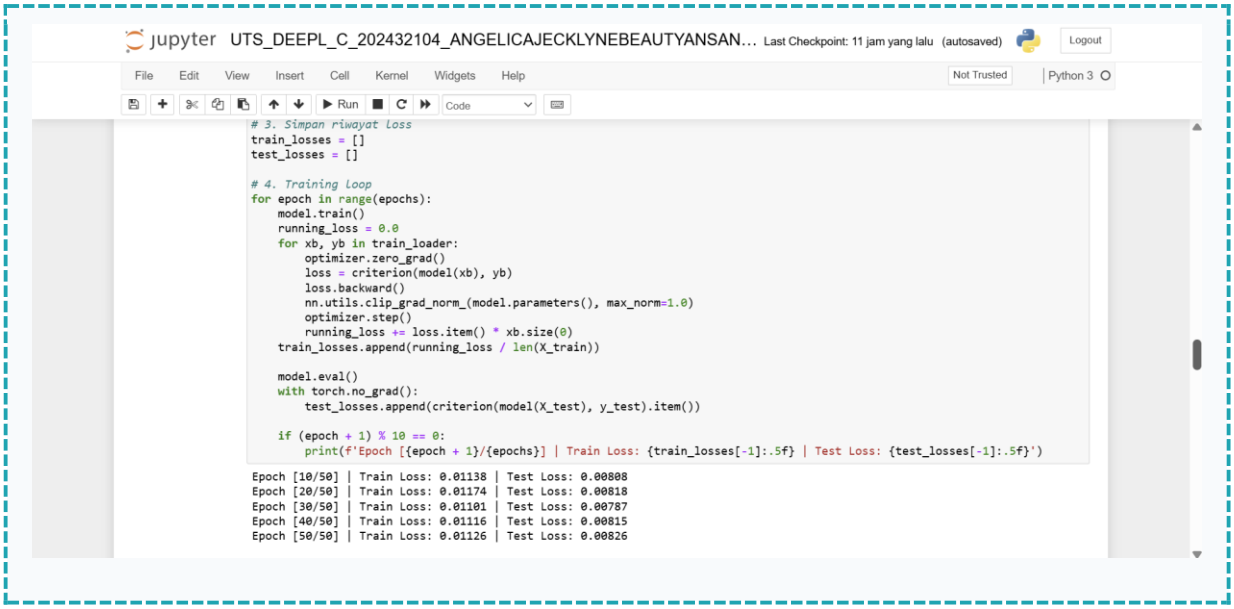
Baris	Kode	Penjelasan
33	test_losses.append(criterion(model(X_test), y_test).item())	Menghitung MSE pada seluruh data uji dengan bobot saat ini lalu menyimpannya. Data uji tidak pernah dipakai untuk memperbarui bobot.
35	if (epoch + 1) % 10 == 0:	Kondisi agar pencetakan hanya dilakukan setiap kelipatan 10 epoch.
36	print(f'Epoch [{epoch + 1}/{epochs}] | Train Loss: {train_losses[-1]:.5f} | Test Loss: {test_losses[-1]:.5f}')	Mencetak nomor epoch, loss latih, dan loss uji dengan lima angka desimal.

Output yang dihasilkan:

```
Epoch [10/50] | Train Loss: 0.01138 | Test Loss: 0.00808
Epoch [20/50] | Train Loss: 0.01174 | Test Loss: 0.00818
Epoch [30/50] | Train Loss: 0.01101 | Test Loss: 0.00787
Epoch [40/50] | Train Loss: 0.01116 | Test Loss: 0.00815
Epoch [50/50] | Train Loss: 0.01126 | Test Loss: 0.00826
```

Analisis. Loss latih telah mendatar di sekitar 0,011 sejak epoch ke-10, dan loss uji stabil di sekitar 0,008. Tidak ada tren kenaikan pada loss uji, sehingga model tidak menunjukkan gejala *overfitting*. Analisis lengkap kurva loss dibahas pada Subbab 3.6.





Gambar 3.4 Screenshot Jupyter: Cell 4 (hyperparameter dan training) beserta log epoch

3.6 Cell 5: Visualisasi Performa Model dan Evaluasi

Cell ini menampilkan grafik riwayat loss, menghitung prediksi pada data uji, mengembalikannya ke skala asli, menghitung metrik evaluasi, dan menampilkan grafik data aktual terhadap prediksi.

Baris	Kode	Penjelasan
1	#202432104_AngelicaJecklyneBeautyAnsanay	Komentar watermark #nim_nama.
2	# 1. Plot Grafik Penurunan Loss Training terhadap Epoch	Komentar bagian 1: grafik penurunan loss.
3	plt.figure(figsize=(10, 4))	Membuat kanvas grafik berukuran 10 × 4 inci.
4	plt.plot(train_losses, color='#114F6A', linewidth=2, label='Training Loss')	Menggambar kurva loss latih terhadap epoch dengan warna biru tua (#114F6A) sesuai warna template.
5	plt.plot(test_losses, color='#1FA4B1', linewidth=2, linestyle='--', label='Test Loss')	Menggambar kurva loss uji dengan warna teal (#1FA4B1) dan garis putus-putus agar mudah dibedakan.
6–8	plt.title('Grafik Penurunan Loss Selama Pelatihan') plt.xlabel('Epoch') plt.ylabel('Loss (MSE, data ternormalisasi)')	Menambahkan judul grafik (baris 6), label sumbu-x “Epoch” (baris 7), dan label sumbu-y yang menjelaskan

Baris	Kode	Penjelasan
		bahwa loss berupa MSE pada data ternormalisasi (baris 8).
9–10	plt.grid(True, linestyle='--', alpha=0.5) plt.legend()	Menampilkan garis kisi putus-putus (baris 9) dan legenda yang memuat nama kurva (baris 10).
11	plt.show()	Menampilkan grafik pada notebook.
13	# 2. Prediksi pada data uji dan kembalikan ke skala asli	Komentar bagian 2: prediksi pada data uji dan pengembalian ke skala asli.
14	model.eval()	Mengaktifkan mode evaluasi.
15	with torch.no_grad():	Menonaktifkan perhitungan gradien selama prediksi.
16	pred_scaled = model(X_test).numpy().flatten()	Memprediksi seluruh 877 sampel uji. .numpy() mengubah tensor menjadi array NumPy dan .flatten() meratakannya menjadi satu dimensi.
17	scale = data_max - data_min	Menghitung rentang normalisasi: 104 − 4 = 100.
18	pred = pred_scaled * scale + data_min	Denormalisasi prediksi (kebalikan Min-Max): pred = pred_scaled × scale + data_min, sehingga kembali ke skala indeks ISPU.
19	actual = y_test.numpy().flatten() * scale + data_min	Denormalisasi nilai aktual data uji dengan cara yang sama.
20	naive = X_test[:, -1, 0].numpy() * scale + data_min	Membuat prediksi *baseline*: elemen terakhir setiap jendela (X_test[:, -1, 0], yaitu nilai kemarin) dipakai sebagai prediksi hari ini, lalu didenormalisasi.
22	def metrics(a, p):	Mendefinisikan fungsi metrics yang menerima nilai aktual a dan prediksi p.
23	mae = np.mean(np.abs(a - p))	MAE: rata-rata nilai absolut selisih.

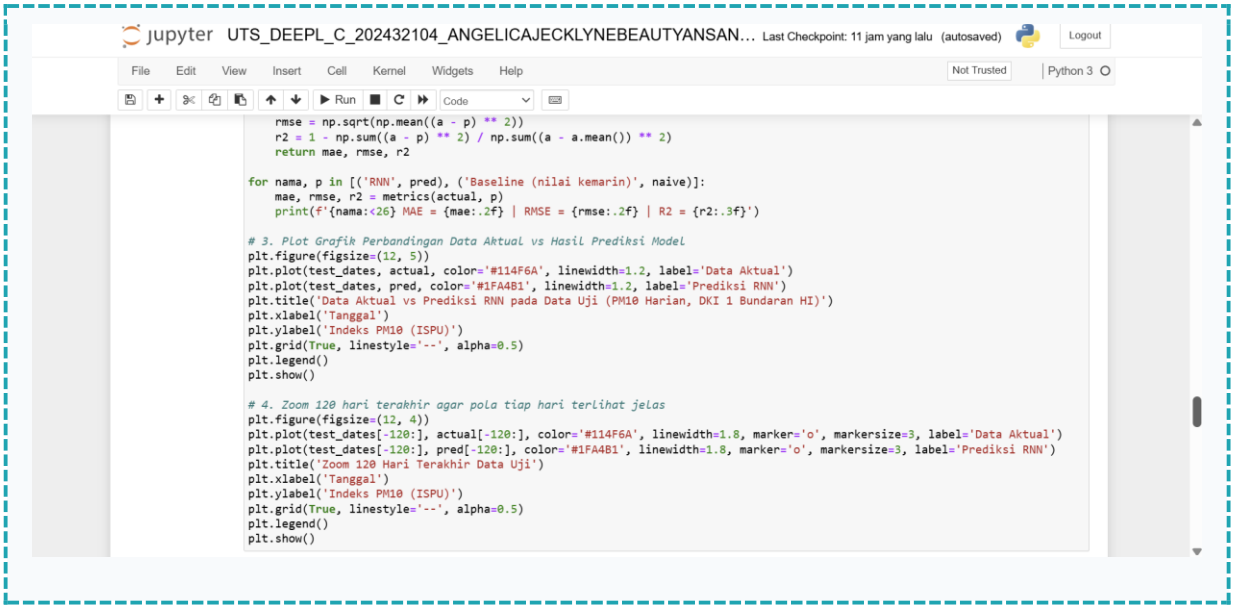
Baris	Kode	Penjelasan
24	rmse = np.sqrt(np.mean((a - p) ** 2))	RMSE: akar dari rata-rata kuadrat selisih.
25	r2 = 1 - np.sum((a - p) ** 2) / np.sum((a - a.mean()) ** 2)	R^2: 1 dikurangi rasio jumlah kuadrat kesalahan model terhadap jumlah kuadrat penyimpangan data dari rata-ratanya.
26	return mae, rmse, r2	Mengembalikan ketiga metrik.
28	for nama, p in [('RNN', pred), ('Baseline (nilai kemarin)', naive)]:	Perulangan atas dua pasangan (nama, prediksi): model RNN dan baseline.
29	mae, rmse, r2 = metrics(actual, p)	Menghitung MAE, RMSE, dan R^2 untuk pasangan tersebut.
30	print(f'{nama:<26} MAE = {mae:.2f} | RMSE = {rmse:.2f} | R2 = {r2:.3f}')	Mencetak hasil dengan format teks: :<26 membuat nama rata kiri selebar 26 karakter, .2f dua desimal, dan .3f tiga desimal.
32	# 3. Plot Grafik Perbandingan Data Aktual vs Hasil Prediksi Model	Komentar bagian 3: grafik data aktual vs prediksi.
33	plt.figure(figsize=(12, 5))	Membuat kanvas grafik berukuran 12 × 5 inci.
34	plt.plot(test_dates, actual, color='#114F6A', linewidth=1.2, label='Data Aktual')	Menggambar data aktual terhadap tanggal (test_dates) dengan warna biru tua.
35	plt.plot(test_dates, pred, color='#1FA4B1', linewidth=1.2, label='Prediksi RNN')	Menggambar prediksi RNN terhadap tanggal dengan warna teal.
36–38	plt.title('Data Aktual vs Prediksi RNN pada Data Uji (PM10 Harian, DKI 1 Bundaran HI)') plt.xlabel('Tanggal') plt.ylabel('Indeks PM10 (ISPU)')	Judul (baris 36), label sumbu-x “Tanggal” (baris 37), dan label sumbu-y “Indeks PM10 (ISPU)” (baris 38).
39–40	plt.grid(True, linestyle='--', alpha=0.5) plt.legend()	Garis kisi (baris 39) dan legenda (baris 40).
41	plt.show()	Menampilkan grafik.
43	# 4. Zoom 120 hari terakhir agar pola tiap hari terlihat jelas	Komentar bagian 4: perbesaran 120 hari terakhir.
44	plt.figure(figsize=(12, 4))	Membuat kanvas 12 × 4 inci.

Baris	Kode	Penjelasan
45	plt.plot(test_dates[-120:], actual[-120:], color='#114F6A', linewidth=1.8, marker='o', markersize=3, label='Data Aktual')	Menggambar data aktual untuk 120 hari terakhir saja ([-120:]) dengan penanda titik (marker='o') agar pergerakan harian terlihat.
46	plt.plot(test_dates[-120:], pred[-120:], color='#1FA4B1', linewidth=1.8, marker='o', markersize=3, label='Prediksi RNN')	Menggambar prediksi RNN untuk 120 hari yang sama dengan penanda titik.
47–49	plt.title('Zoom 120 Hari Terakhir Data Uji') plt.xlabel('Tanggal') plt.ylabel('Indeks PM10 (ISPU)')	Judul (baris 47), label sumbu-x (baris 48), dan label sumbu-y (baris 49).
50–51	plt.grid(True, linestyle='--', alpha=0.5) plt.legend()	Garis kisi (baris 50) dan legenda (baris 51).
52	plt.show()	Menampilkan grafik.

Output teks yang dihasilkan:

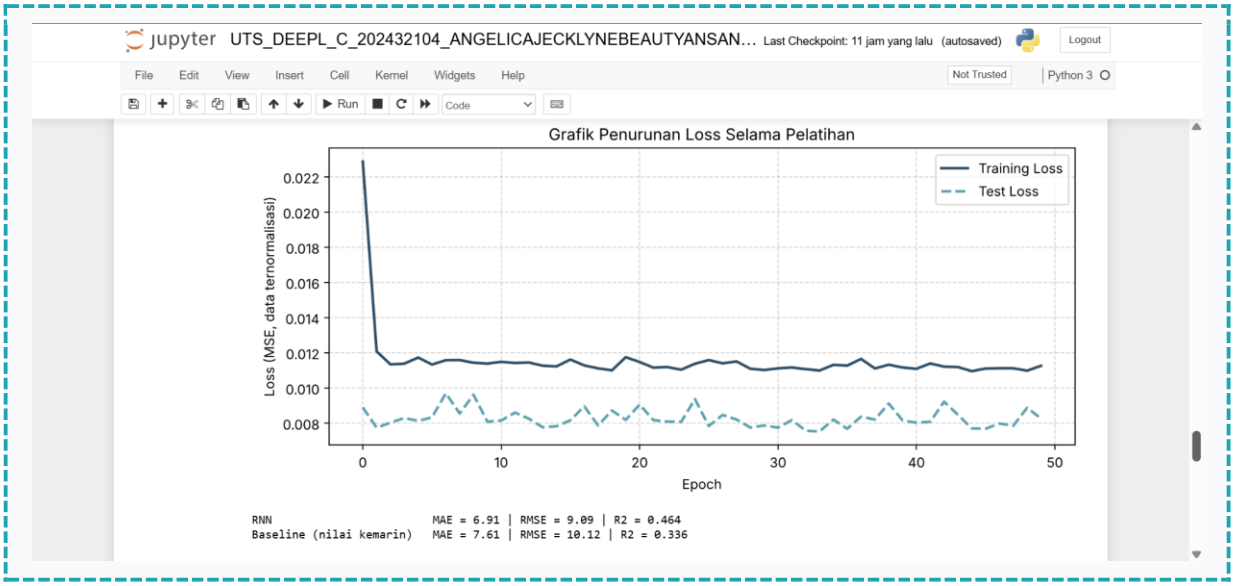
```
RNN                    MAE = 6.91 | RMSE = 9.09 | R2 = 0.464
Baseline (nilai kemarin)  MAE = 7.61 | RMSE = 10.12 | R2 = 0.336
```





Gambar 3.5 Screenshot Jupyter: Cell 5 beserta output teks metrik evaluasi

3.6.1 Grafik riwayat training (loss terhadap epoch)



Gambar 3.6 Screenshot Jupyter: grafik penurunan loss selama pelatihan

Analisis. Loss latih turun sangat tajam dari 0,0229 pada epoch pertama menjadi 0,0121 pada epoch kedua, lalu mendatar di kisaran 0,011–0,0117 sampai epoch ke-50. Artinya, model sudah konvergen sejak awal dan tambahan epoch tidak memberi perbaikan berarti; 50 epoch lebih dari cukup. Fluktuasi kecil pada kurva latih wajar karena laju pembelajaran 0,01 dan *mini-batch* yang diacak.

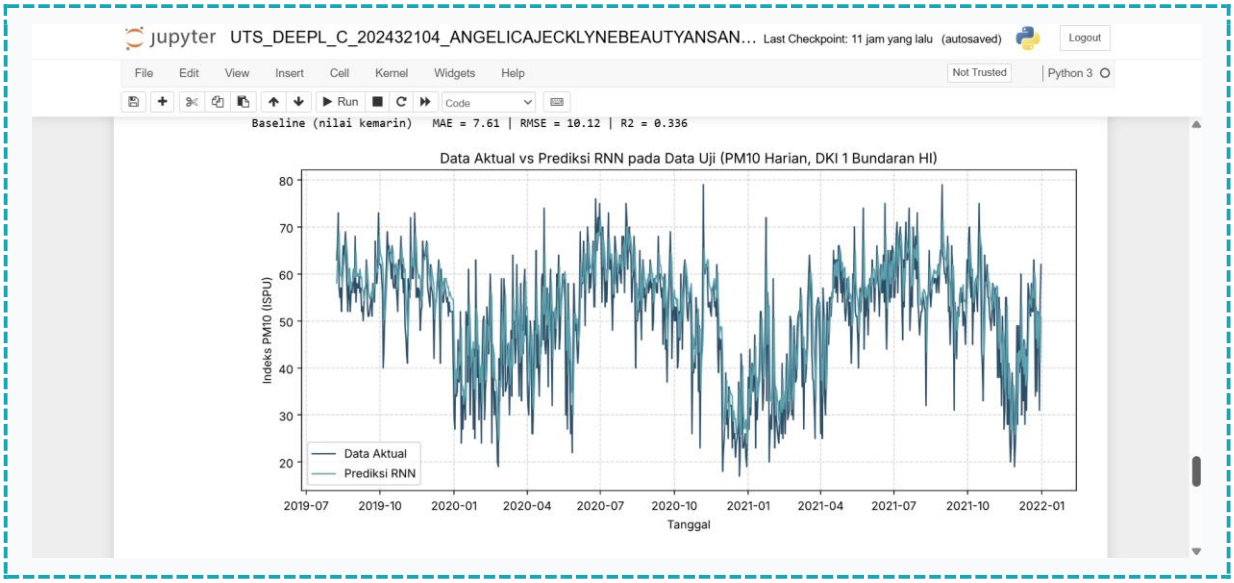
Loss uji berfluktuasi di antara 0,0075 dan 0,0097 tanpa tren naik. Karena tidak ada jarak yang melebar di mana loss latih terus turun sementara loss uji naik, tidak ada tanda *overfitting*. Loss uji justru lebih rendah daripada loss latih. Hal ini disebabkan periode uji (8

Agustus 2019 s.d. 31 Desember 2021) lebih tenang, dengan simpangan baku 12,4 dibandingkan 15,0 pada periode latih, sehingga lebih mudah ditebak. Penyebabnya bukan kebocoran data, karena normalisasi hanya memakai statistik data latih. Sebagai pemeriksaan, akar dari loss uji akhir (0,00826) dikalikan rentang normalisasi 100 memberi 9,09, sama dengan RMSE pada skala asli.

3.6.2 Grafik data aktual vs prediksi

Tabel 3.2 Perbandingan metrik pada data uji (877 hari)

Model	MAE	RMSE	R^2	Perbaikan
RNN	**6,91**	**9,09**	**0,464**	RMSE turun 10,2% dan MAE turun 9,2% dari baseline
Baseline (nilai kemarin)	7,61	10,12	0,336	-

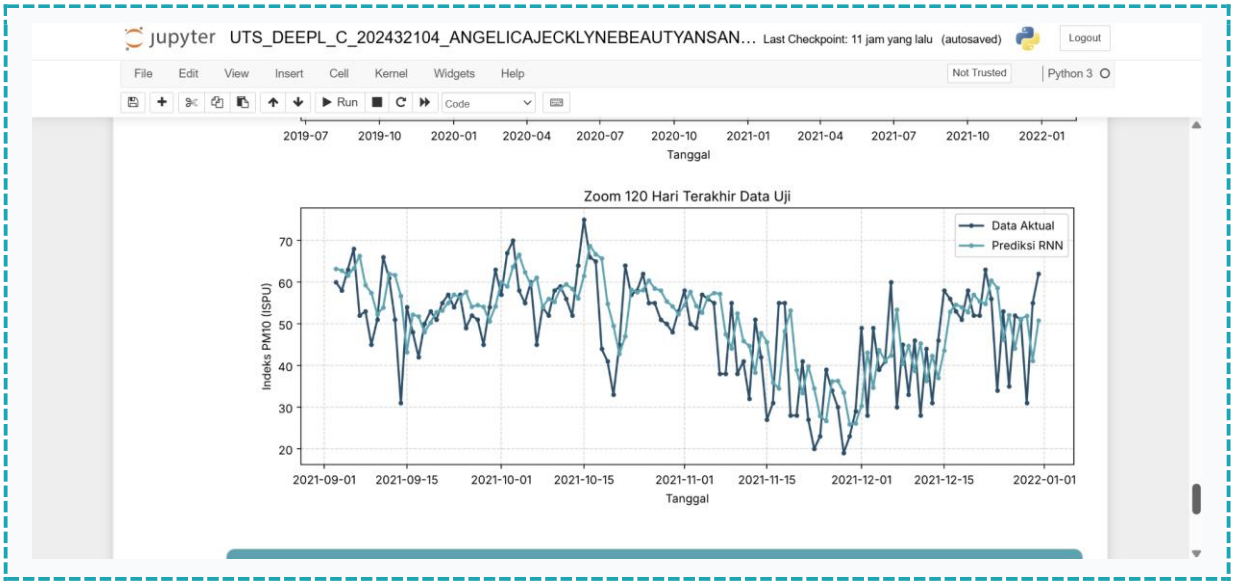


Gambar 3.7 Screenshot Jupyter: grafik perbandingan data aktual vs prediksi RNN pada data uji

Analisis. Model RNN berhasil mengalahkan baseline: RMSE turun dari 10,12 menjadi 9,09 dan R^2 naik dari 0,336 menjadi 0,464. Artinya, model mempelajari pola dari 30 hari terakhir, bukan sekadar menyalin nilai kemarin. Nilai R^2 sebesar 0,464 berarti sekitar 46% variasi PM10 pada data uji dapat dijelaskan model, sehingga performanya tergolong sedang.

Pada grafik, kurva prediksi (teal) mengikuti pergerakan jangka menengah data aktual (biru tua): penurunan pada awal 2020, kenaikan pada pertengahan 2020, penurunan dari akhir 2020 sampai awal 2021, lalu kenaikan hingga pertengahan 2021 dan penurunan pada akhir

2021. Namun, prediksi cenderung lebih halus daripada data aktual. Puncak yang tinggi (misalnya sekitar 79 pada November 2020 dan Agustus 2021) hanya diprediksi sekitar 65–72, sedangkan lembah terdalam (sekitar 17–20) diprediksi sekitar 25–30. Kecenderungan ini wajar pada model yang meminimalkan MSE.



Gambar 3.8 Screenshot Jupyter: grafik perbesaran 120 hari terakhir data uji

Analisis. Pada grafik perbesaran, terlihat bahwa prediksi sering **tertinggal sekitar satu hari** terhadap perubahan mendadak. Contohnya lonjakan ke 75 pada pertengahan Oktober 2021: prediksi pada hari lonjakan hanya sekitar 61 dan baru mencapai sekitar 69 pada hari berikutnya. Demikian pula saat nilai aktual anjlok pada hari-hari sesudahnya, prediksi masih tinggi. Pola ini umum pada peramalan harian yang hanya memakai riwayat nilai itu sendiri: model menebak hari berikutnya dari kecenderungan hari-hari sebelumnya, sehingga tidak dapat mengantisipasi guncangan yang dipicu faktor lain.

Keterbatasan utama adalah model hanya memakai satu variabel (PM10 masa lalu) tanpa data cuaca, kelembapan, arah angin, atau polutan lain, serta RNN sederhana yang tidak memiliki mekanisme gerbang seperti LSTM. Perbaikan yang dapat dicoba adalah menambah fitur eksternal, menguji LSTM atau GRU, serta menyetel panjang jendela dan jumlah neuron.

BAB 4
KESIMPULAN TEORI DAN PRAKTIKUM

Bab ini merangkum keterkaitan antara konsep teori dan perhitungan manual pada Bab 2 dengan hasil implementasi dan visualisasi performa model pada Bab 3.

4.1 Keterkaitan Teori dan Implementasi

1. **Rumus yang sama.** Perhitungan manual pada Bab 2 menggunakan $h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$ dan menghasilkan $h_t = 0{,}5703$. Rumus yang sama dijalankan oleh nn.RNN pada Cell 3. Verifikasi dengan PyTorch memberi hasil 0,57032, sehingga pemahaman alur komputasi satu langkah waktu terbukti benar.
2. **Dari satu neuron ke model penuh.** Perhitungan manual memakai satu neuron dan satu langkah waktu. Model pada implementasi menjalankan sel yang sama untuk 64 neuron selama 30 langkah, dengan hidden state awal nol, lalu mengambil hidden state langkah ke-30 untuk diteruskan ke lapisan linear (Gambar 2.2 dan Cell 3, baris 9–11).
3. **Normalisasi sama.** Langkah normalisasi pada perhitungan manual ($x' = (x - 4) / (104 - 4)$) identik dengan Cell 2 baris 20, dengan $x_{min} = 4$ dan $x_{max} = 104$ yang dihitung dari data latih. Denormalisasi pada Cell 5 baris 18 membalik proses itu agar hasil dibaca dalam skala ISPU.
4. **Mekanisme pelatihan.** *Forward propagation*, MSE, *backpropagation through time*, *gradient clipping*, dan optimizer Adam yang dijelaskan pada Subbab 2.5 diimplementasikan pada Cell 4 baris 24–27. Kurva loss pada Gambar 3.6 menunjukkan hasilnya: loss turun cepat dan stabil tanpa tanda *overfitting*.

4.2 Kesimpulan

1. RNN dapat dipakai untuk meramalkan indeks PM10 harian di Stasiun DKI 1 Bundaran HI. Dengan jendela 30 hari dan 64 neuron tersembunyi (4.353 parameter), model mencapai **MAE 6,91, RMSE 9,09, dan R^2 0,464** pada 877 hari data uji.
2. Model lebih baik daripada baseline "nilai kemarin" (RMSE 10,12 dan R^2 0,336), dengan penurunan RMSE sekitar 10%. Ini menunjukkan model benar-benar memanfaatkan informasi urutan dari 30 hari sebelumnya.
3. Prediksi mengikuti tren jangka menengah, tetapi puncak dan lembah diratakan serta perubahan mendadak terlambat sekitar satu hari. Dengan R^2 sekitar 0,46, performa model tergolong sedang dan belum cukup untuk peramalan kejadian ekstrem.

4. Kualitas data memengaruhi desain pemodelan: kolom PM2,5 yang 77,8% kosong tidak dipakai, dan karena seluruh tahun 2022 tidak memiliki data, pemodelan memakai segmen kontinu 2010–2021.

4.3 Saran

Pengembangan berikutnya dapat dilakukan dengan menambahkan variabel pendukung seperti cuaca dan polutan lain sebagai fitur multivariat, membandingkan RNN dengan LSTM atau GRU, menyetel panjang jendela dan hyperparameter, serta memakai data stasiun lain atau data tahun 2023–2025 untuk menguji kemampuan generalisasi model.